Table of Contents

Introduction **2**

Update the Database to Include Identity **3**

Add Authentication **6**

Install Required Packages *6*

Add ViewModels and Views *6*

Add AccountController.cs *9*

Add Login Navigation, Partial Views *12*

Update Program.cs *14*

Test the App *14*

Add Authorisation **15**

Demo: Authentication and Authorisation

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT and Gemini.

Introduction

This demo guides you through adding authentication and authorisation to our ASP.NET Core application using a clean, controlled coding approach. Authentication allows shop management users to register and log in, while authorisation prevents unauthenticated users from modifying database content. Instead of relying on auto-generated scaffolding that hides logic, this approach builds the authentication flow manually for complete visibility.

The demo includes:

- **Update the Database to Include Identity:** Installing Entity Framework Core Identity packages, upgrading the DbContext, and running migrations to generate the underlying user tables.
- **Add Authentication via Controlled Coding:** Creating custom ViewModels, Razor Views, and an AccountController that leverages the built-in UserManager and SignInManager services.
- **UI Integration & Middleware:** Updating the application layout with navigation partial views, enabling client-side validation scripts, and configuring session and cookie middleware in Program.cs.
- **Add Authorisation:** Securing specific controller actions using the [Authorize] attribute to restrict database modifications to logged-in users.
- **Password Security & Architecture:** Understanding how ASP.NET Core Identity automatically handles secure cryptographic hashing and salting, and why this controlled architecture protects against common web vulnerabilities.

Update the Database to Include Identity

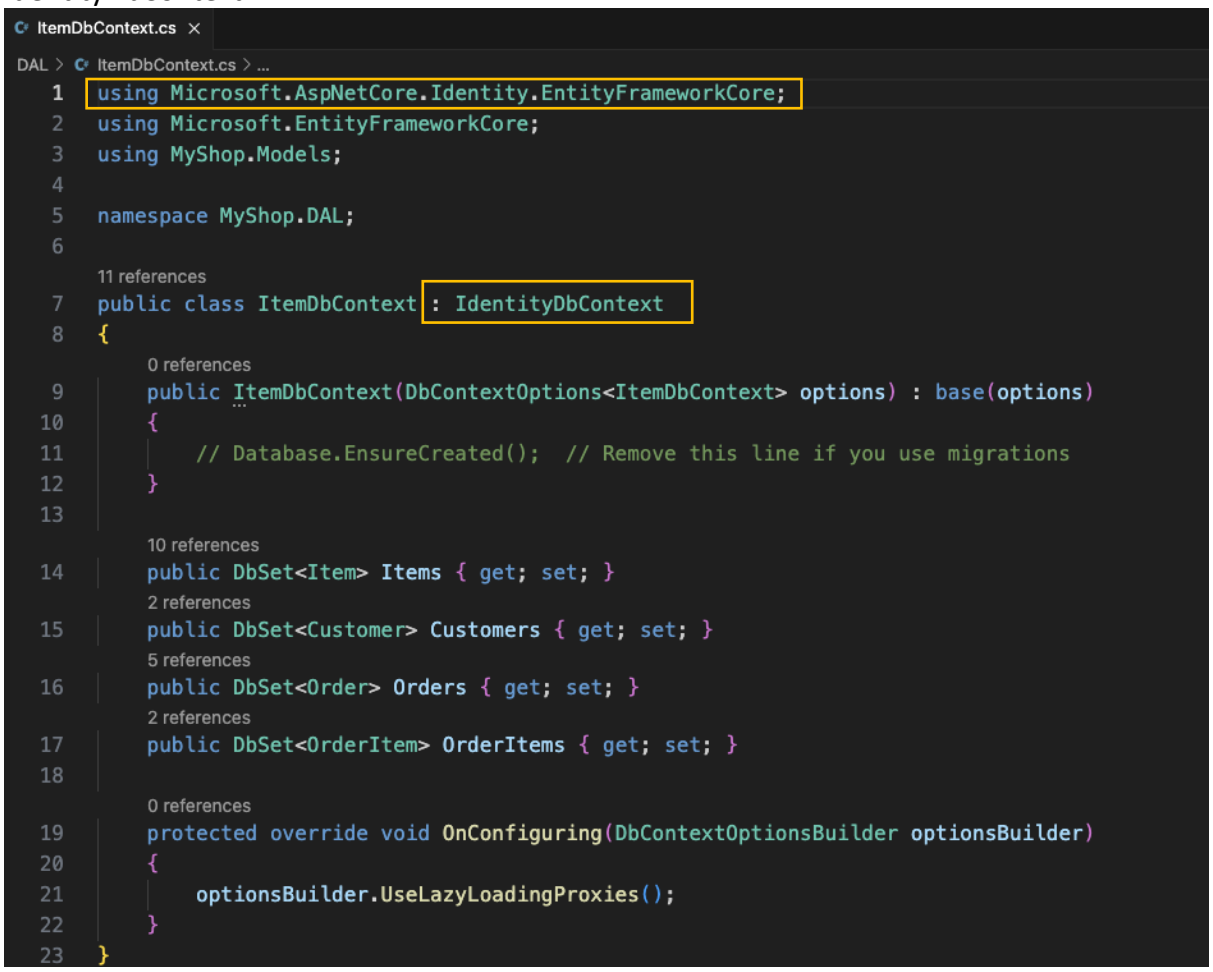
Step 1: Install Identity Packages

Install the required ASP.NET Core Identity packages matching .NET 10:

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore --version "10.*"
dotnet add package Microsoft.AspNetCore.Identity.UI --version "10.*"
```

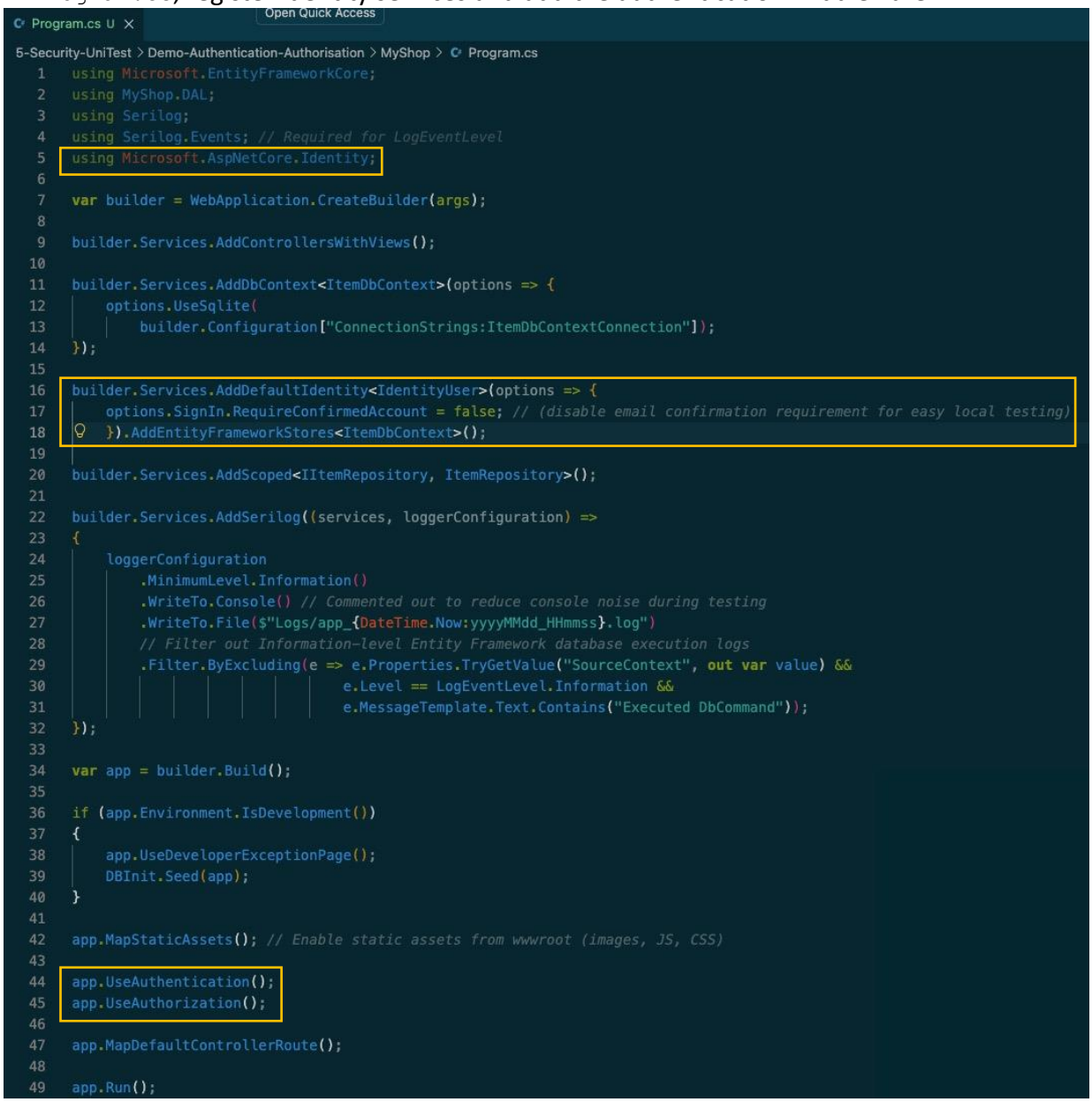
Step 2: Update the DbContext

In DAL/ItemDbContext.cs, change the base class of ItemDbContext from DbContext to IdentityDbContext:



Step 3: Register Identity & Add Authentication Middleware

In `Program.cs`, register Identity services and add the authentication middleware:



Step 4: Update the Database

Choose **Option A** or **Option B** depending on how your database lifecycle is managed:

- **Option A: Using Database Re-creation (Development/Testing)** If your seeding logic uses context.Database.EnsureDeleted() and context.Database.EnsureCreated(), simply build and run the application to apply the new schema.

Note: EnsureDeleted() completely drops and recreates the database on startup. This is convenient during early development, but **never use this in production** as it destroys existing data.

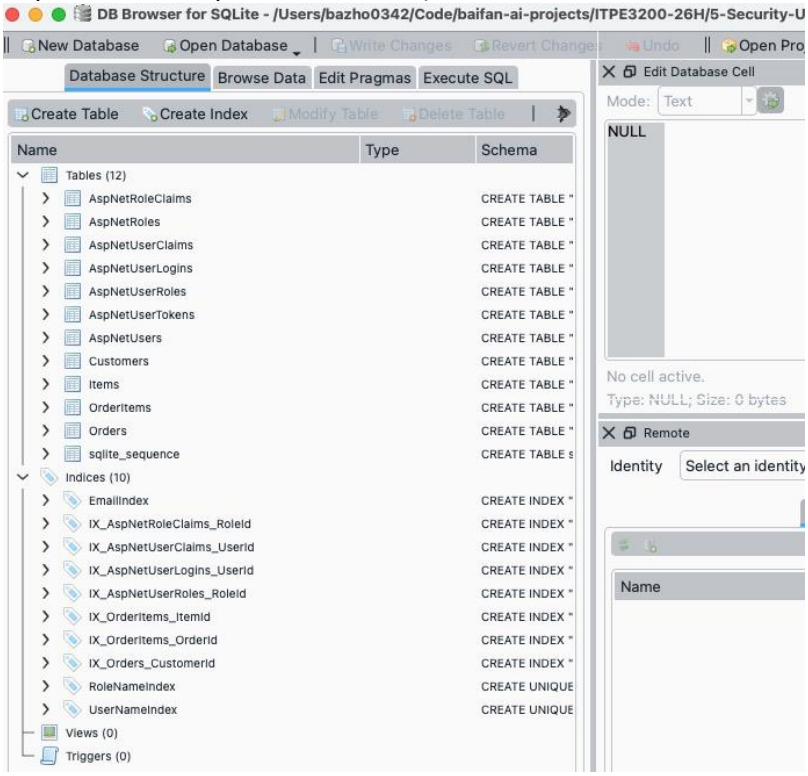
- **Option B: Using EF Core Migrations (Production/Preserving data)** If you track database changes with EF Core Migrations, run the following commands in the terminal:

```
dotnet ef migrations add IdentityAdded
dotnet ef database update
```

Note: If your initial database was created using EnsureCreated() instead of migrations, running database update will fail with a "*Table already exists*" error. Migrations require the entire database lifecycle to be managed via dotnet ef database update.

Step 5: Verify the Schema

Build and run the project. If you open the SQLite database using **DB Browser for SQLite**, you will now see the newly generated ASP.NET Core Identity tables (e.g., AspNetUsers, AspNetRoles, AspNetUserClaims).



Add Authentication

Instead of using full scaffolding (which generates many unnecessary files in an `Areas/` folder), we will manually configure ASP.NET Core Identity to maintain clean, fully controlled code.

Install Required Packages

Install EF Core tools and ASP.NET Core Identity packages matching .NET 10:

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore --version "10.*"
dotnet add package Microsoft.EntityFrameworkCore.Tools --version "10.*"
```

- **Package Resolution Tip:** If you encounter a package downgrade or compatibility error (e.g., NU1605), explicitly install the matching Microsoft.EntityFrameworkCore.Design package version first, then re-run the Tools package installation:

```
dotnet add package Microsoft.EntityFrameworkCore.Design --version "10.*"
```

Add ViewModels and Views

Create the necessary classes and Razor views to handle user registration, login, and logout:

Step 1 ViewModels: Create ViewModels/RegisterViewModel.cs and ViewModels/LoginViewModel.cs to hold user inputs and data annotation validation rules (including ReturnUrl support for redirecting to the page you came from after login).

LoginViewModel.cs

```
using System.ComponentModel.DataAnnotations;

namespace MyShop.ViewModels;

public class LoginViewModel
{
    [Required]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; } = string.Empty;

    [Display(Name = "Remember me?")]
    public bool RememberMe { get; set; }

    // Strongly-typed ReturnUrl property
    public string? ReturnUrl { get; set; }
}
```

RegisterViewModel.cs

```
using System.ComponentModel.DataAnnotations;

namespace MyShop.ViewModels;

public class RegisterViewModel
{
    [Required]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; } = string.Empty;

    [DataType(DataType.Password)]
    [Display(Name = "Confirm password")]
    [Compare("Password", ErrorMessage = "The password and confirmation password do not match.")]
    public string ConfirmPassword { get; set; } = string.Empty;
}
```

Step 2 Views: Create Views/Account/Register.cshtml and Views/Account/Login.cshtml with input forms bound to their respective ViewModels.

Login.cshtml U

5-Security-UniTest > Demo-Authentication-Authorisation-2 > MyShop > Views > Account > Login.cshtml

```cshtml
@model MyShop.ViewModels.LoginViewModel

<h2>Login</h2>
<div class="row">
    <div class="col-md-6">
        <form asp-action="Login" method="post">
            <!-- Hidden input preserves ReturnUrl across form posts -->
            <input type="hidden" asp-for="ReturnUrl" />

            <div asp-validation-summary="ModelOnly" class="text-danger"></div>

            <div class="mb-3">
                <label asp-for="Email" class="form-label"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>

            <div class="mb-3">
                <label asp-for="Password" class="form-label"></label>
                <input asp-for="Password" class="form-control" />
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>

            <div class="mb-3 form-check">
                <input asp-for="RememberMe" class="form-check-input" />
                <label asp-for="RememberMe" class="form-check-label"></label>
            </div>

            <button type="submit" class="btn btn-primary">Log in</button>
        </form>
    </div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Register.cshtml U

5-Security-UniTest > Demo-Authentication-Authorisation-2 > MyShop > Views > Account > Register.cshtml

```cshtml
@model MyShop.ViewModels.RegisterViewModel

<h2>Register</h2>
<div class="row">
    <div class="col-md-6">
        <form asp-action="Register" method="post">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="mb-3">
                <label asp-for="Email" class="form-label"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="mb-3">
                <label asp-for="Password" class="form-label"></label>
                <input asp-for="Password" class="form-control" />
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <div class="mb-3">
                <label asp-for="ConfirmPassword" class="form-label"></label>
                <input asp-for="ConfirmPassword" class="form-control" />
                <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
            </div>
            <button type="submit" class="btn btn-primary">Register</button>
        </form>
    </div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Step 3 Validation Scripts Partial: Notice that both Register and Login Views need _ValidationScriptsPartial.cshtml. Create Views/Shared/_ValidationScriptsPartial.cshtml to load jQuery validation libraries.

_ValidationScriptsPartial.cshtml U

5-Security-UniTest > Demo-Authentication-Authorisation-2 > MyShop > Views > Shared > _ValidationScriptsPartial.cshtml

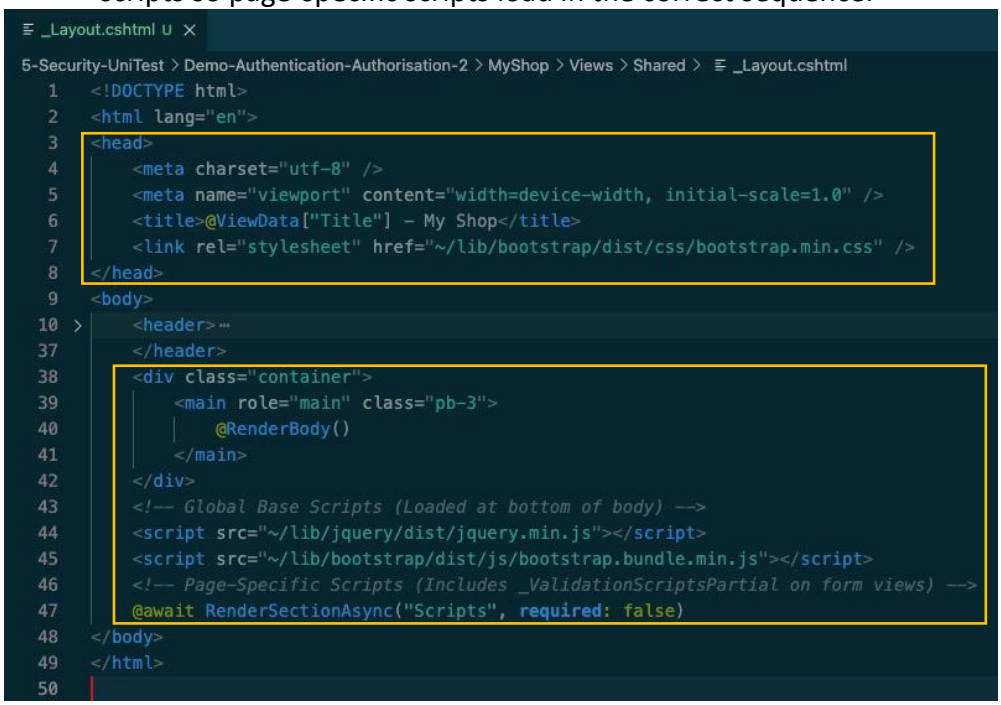
```cshtml
<script src="~/lib/jquery-validation/dist/jquery.validate.min.js"></script>
<script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js"></script>
```

Optional: Optimising Client-Side Script Performance
In the initial setup, client-side validation libraries were included globally in _Layout.cshtml for simplicity. However, loading these scripts on every page, including read-only listing or detail pages, creates unnecessary network overhead.

To optimise application performance and follow standard ASP.NET Core MVC architecture, we will update _Layout.cshtml to load validation libraries conditionally only on pages containing interactive forms.

Update _Layout.cshtml

- Remove the static validation script references (jquery.validate and jquery.validate-unobtrusive) from the <head> section.
- Move the core scripts (jquery and bootstrap) to the bottom of the page, directly above the closing </body> tag.
- Ensure @await RenderSectionAsync("Scripts", required: false) is placed after the core scripts so page-specific scripts load in the correct sequence.



Enable Validation Scripts on Form Views
Add the @section Scripts block to the bottom of any view that contains a form requiring client-side validation:

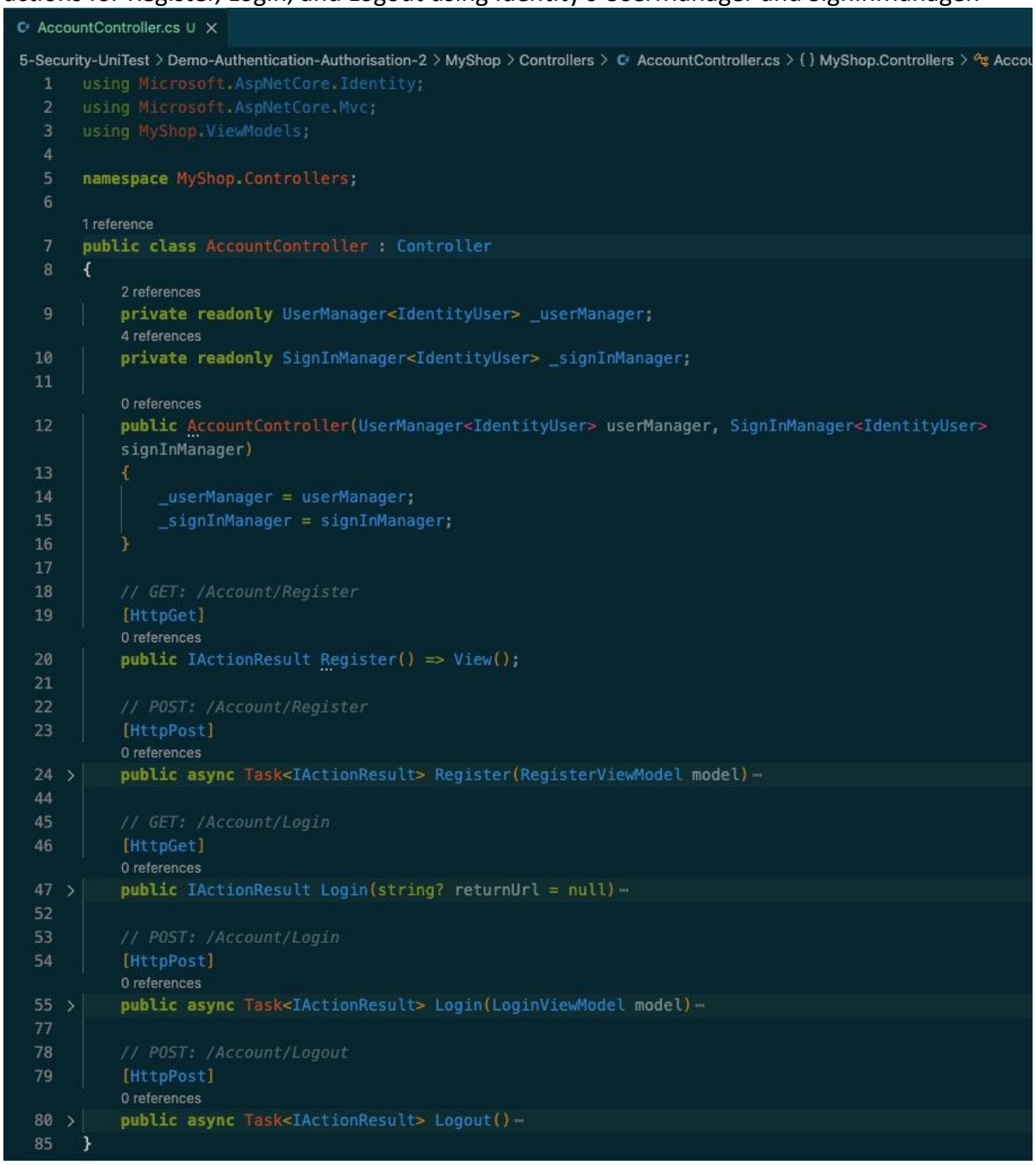
- Views/Account/Register.cshtml (already added)
- Views/Account/Login.cshtml (already added)
- Views/Item/Create.cshtml
- Views/Item/Update.cshtml
- Views/Order/CreateOrderItem.cshtml

```
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Now we only load the validation scripts where it is needed.

Add AccountController.cs

AccountController: Create Controllers/AccountController.cs to handle HTTP GET and POST actions for Register, Login, and Logout using Identity's UserManager and SignInManager.



Instead of manually writing database queries for user accounts, hashing passwords, or managing authentication cookies, we delegate these responsibilities to two built-in framework services injected via Dependency Injection:

UserManager<IdentityUser> Manages user identity data in the database.

- **Why:** Automatically handles secure password hashing (PBKDF2), validation rules (length, special characters), and identity error reporting without manual SQL or repository logic.

SignInManager<IdentityUser>: Handles authentication status and HTTP context (session & Cookie).

- **Why:** Creates, encrypts, and issues authentication cookies upon login, clears them on logout, and provides security defenses against open redirect vulnerabilities and brute-force attacks.

Registration Flow (Register):

- CreateAsync(user, password) hashes the password and persists the new user to SQLite via EF Core.
- SignInAsync(user, isPersistent: false) automatically logs the user in immediately after successful registration.

```
22      // POST: /Account/Register
23      [HttpPost]
        0 references
24      public async Task<IActionResult> Register(RegisterViewModel model)
25      {
26          if (ModelState.IsValid)
27          {
28              var user = new IdentityUser { UserName = model.Email, Email = model.Email };
29              var result = await _userManager.CreateAsync(user, model.Password);
30
31              if (result.Succeeded)
32              {
33                  await _signInManager.SignInAsync(user, isPersistent: false);
34                  return RedirectToAction("Index", "Home");
35              }
36
37              foreach (var error in result.Errors)
38              {
39                  ModelState.AddModelError(string.Empty, error.Description);
40              }
41          }
42          return View(model);
43      }
```

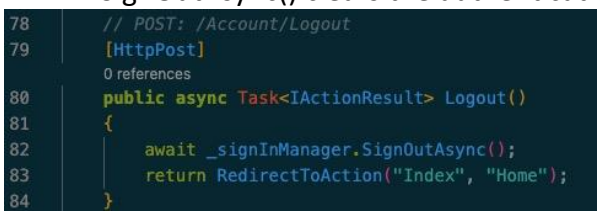
Login Flow (Login):

- Accepts a ReturnUrl to preserve where the user was originally trying to go (e.g., /Item/Create).
- PasswordSignInAsync(...) checks credentials, validates the hash, and sets the auth cookie.
- Url.IsLocalUrl(...) prevents security exploits by ensuring the user is only redirected to internal application routes.

```
45      // GET: /Account/Login
46      [HttpGet]
        0 references
47      public IActionResult Login(string? returnUrl = null)
48      {
49          var model = new LoginViewModel { ReturnUrl = returnUrl };
50          return View(model);
51      }
52
53      // POST: /Account/Login
54      [HttpPost]
        0 references
55      public async Task<IActionResult> Login(LoginViewModel model)
56      {
57          // Default to home page if ReturnUrl is null or empty
58          string targetUrl = !string.IsNullOrEmpty(model.ReturnUrl) && Url.IsLocalUrl(model.ReturnUrl)
59              ? model.ReturnUrl
60              : Url.Action("Index", "Home") ?? "~/";
61
62          if (ModelState.IsValid)
63          {
64              var result = await _signInManager.PasswordSignInAsync(
65                  model.Email, model.Password, model.RememberMe, lockoutOnFailure: false);
66
67              if (result.Succeeded)
68              {
69                  return Redirect(targetUrl);
70              }
71
72              ModelState.AddModelError(string.Empty, "Invalid login attempt.");
73          }
74
75          return View(model);
76      }
```

Logout Flow (Logout):

- SignOutAsync() clears the authentication cookie from the user's browser session.

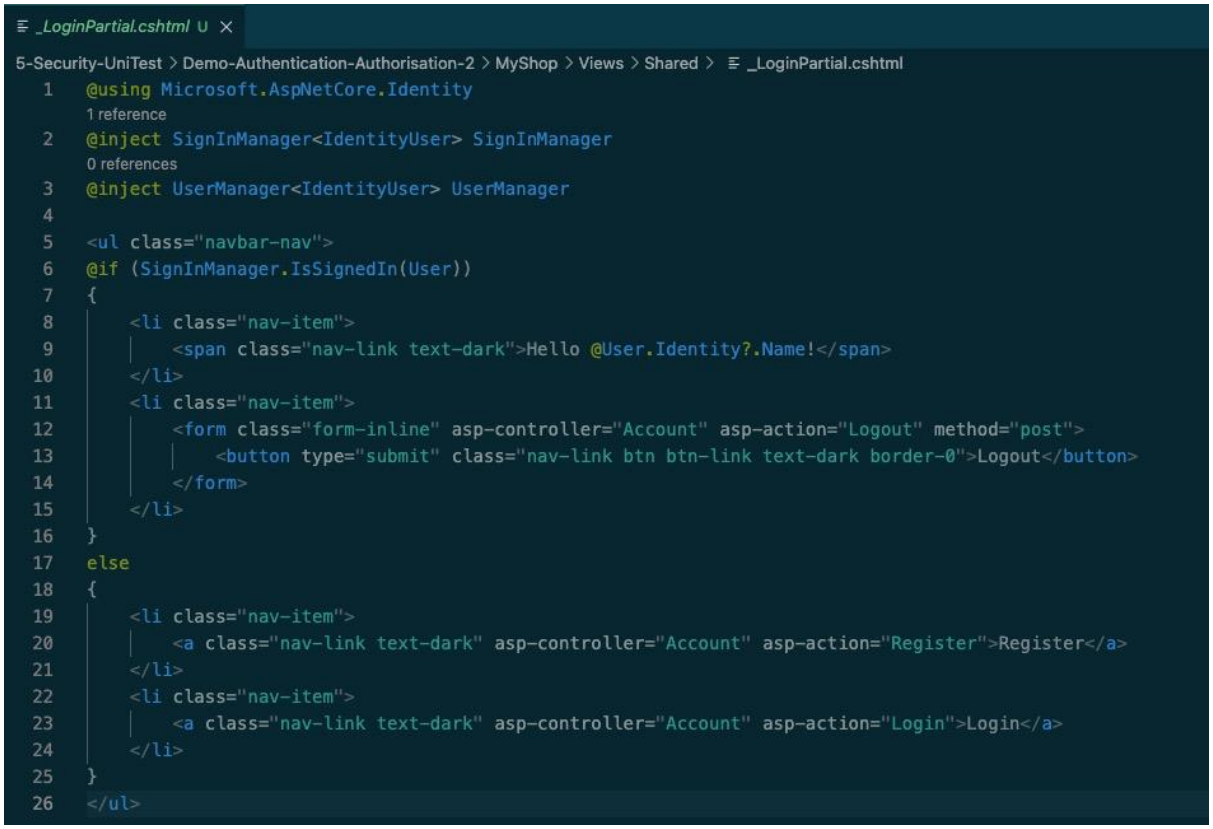


```
78      // POST: /Account/Logout
79      [HttpPost]
        0 references
80      public async Task<IActionResult> Logout()
81      {
82          await _signInManager.SignOutAsync();
83          return RedirectToAction("Index", "Home");
84      }
```

Add Login Navigation, Partial Views

Integrate authentication navigation controls and client-side validation scripts into your application layout:

Step 1 Login Partial View: Create Views/Shared/_LoginPartial.cshtml to conditionally render **Register/Login** links for guests (line17 – line25), or a **User Greeting/Logout** (line7 – line 16) form for authenticated users.



Step 2 Update Layout:

- In Views/Shared/_Layout.cshtml, render _LoginPartial inside your navigation bar.
- **Render Scripts Section:** Ensure _Layout.cshtml includes @await RenderSectionAsync("Scripts", required: false) near the bottom (</body>) to enable client-side form validation.

_Layout.cshtml U

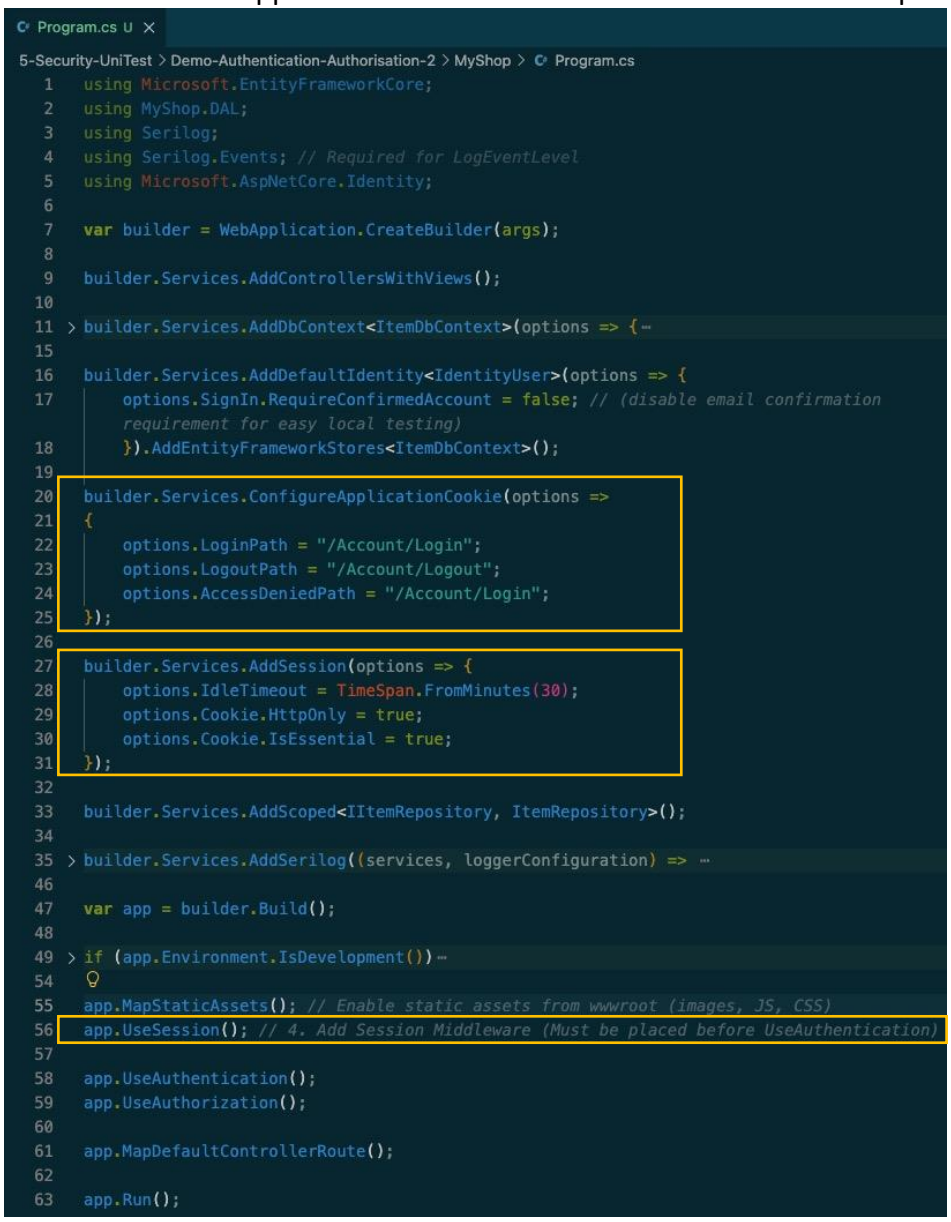
5-Security-UniTest > Demo-Authentication-Authorisation-2 > MyShop > Views > Shared > _Layout.cshtml

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - My Shop</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <script src="~/lib/jquery/dist/jquery.min.js" defer></script>
    <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js" defer></script>
    <script src="~/lib/jquery-validation/jquery.validate.min.js" defer></script>
    <script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js" defer></script>
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom
        box-shadow mb-3">
            <div class="container-fluid">
                <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">My Shop</a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".
                navbar-collapse" aria-controls="navbarSupportedContent"
                        aria-expanded="false" aria-label="Toggle navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>
                <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                    <ul class="navbar-nav flex-grow-1">
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home"
                            asp-action="Index">Home</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Item"
                            asp-action="Table">Items</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Customer"
                            asp-action="Table">Customers</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Order"
                            asp-action="Table">Orders</a>
                        </li>
                    </ul>
                    <partial name="_LoginPartial" />
                </div>
            </div>
        </nav>
    </header>
    <div class="container">
        <main role="main" class="pb-3">
            @RenderBody()
            @await RenderSectionAsync("Scripts", required: false)
        </main>
    </div>
</body>
</html>
```

Update Program.cs

Configure cookie handling, and middleware in your application pipeline:

- **Configure Application Cookie:** Configure application cookie paths to redirect unauthenticated users to /Account/Login and /Account/Logout.
- **Register and Use Session State:** Add builder.Services.AddSession() and register app.UseSession() before authentication.
- *Why Session Support?* HTTP is stateless by default. Enabling session middleware allows the application to retain stateful user data across multiple HTTP requests.



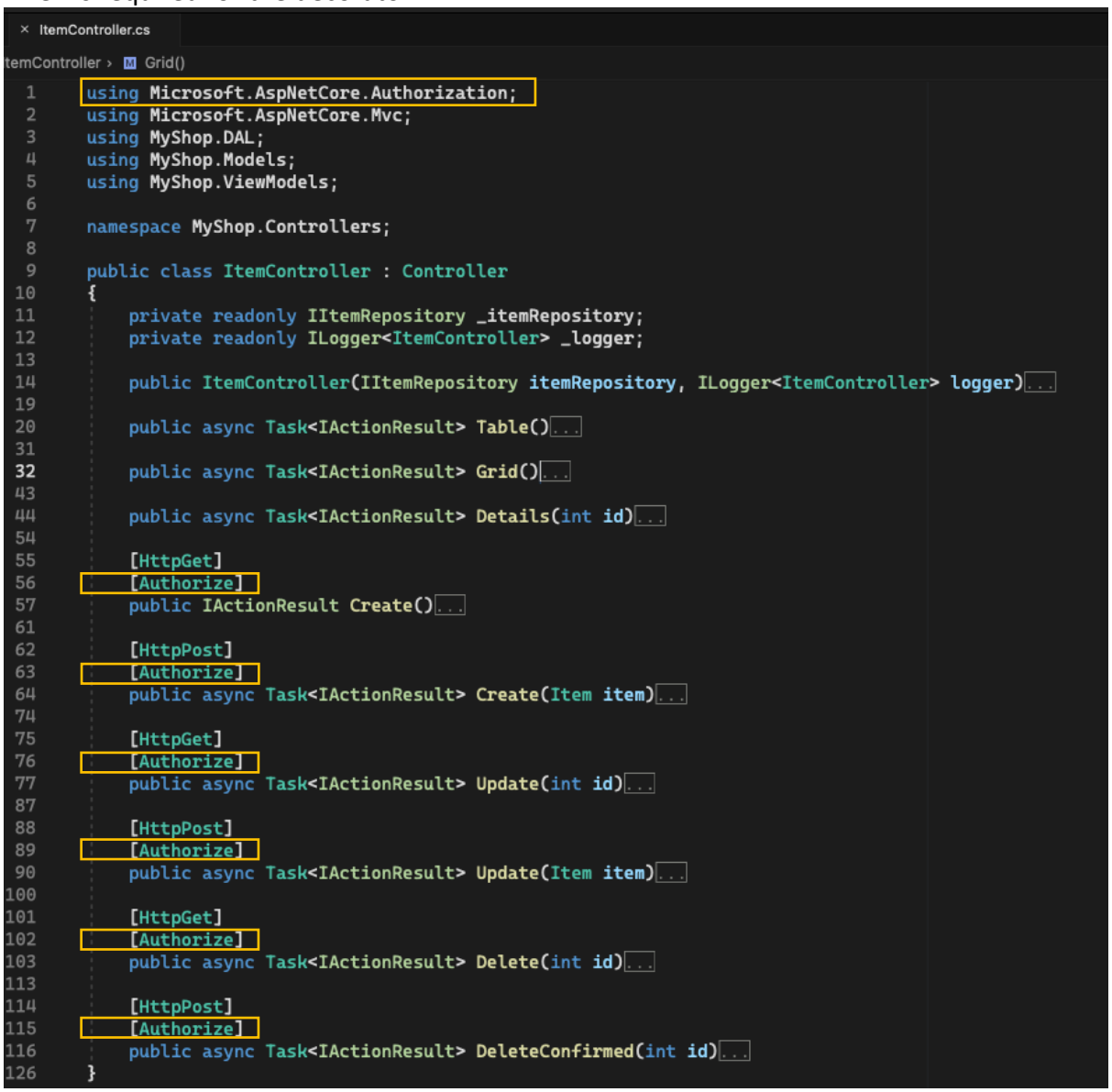
Test the App

Navigate to the register and login page to test them. Built-in model state validation will automatically handle password complexity and required field checks.

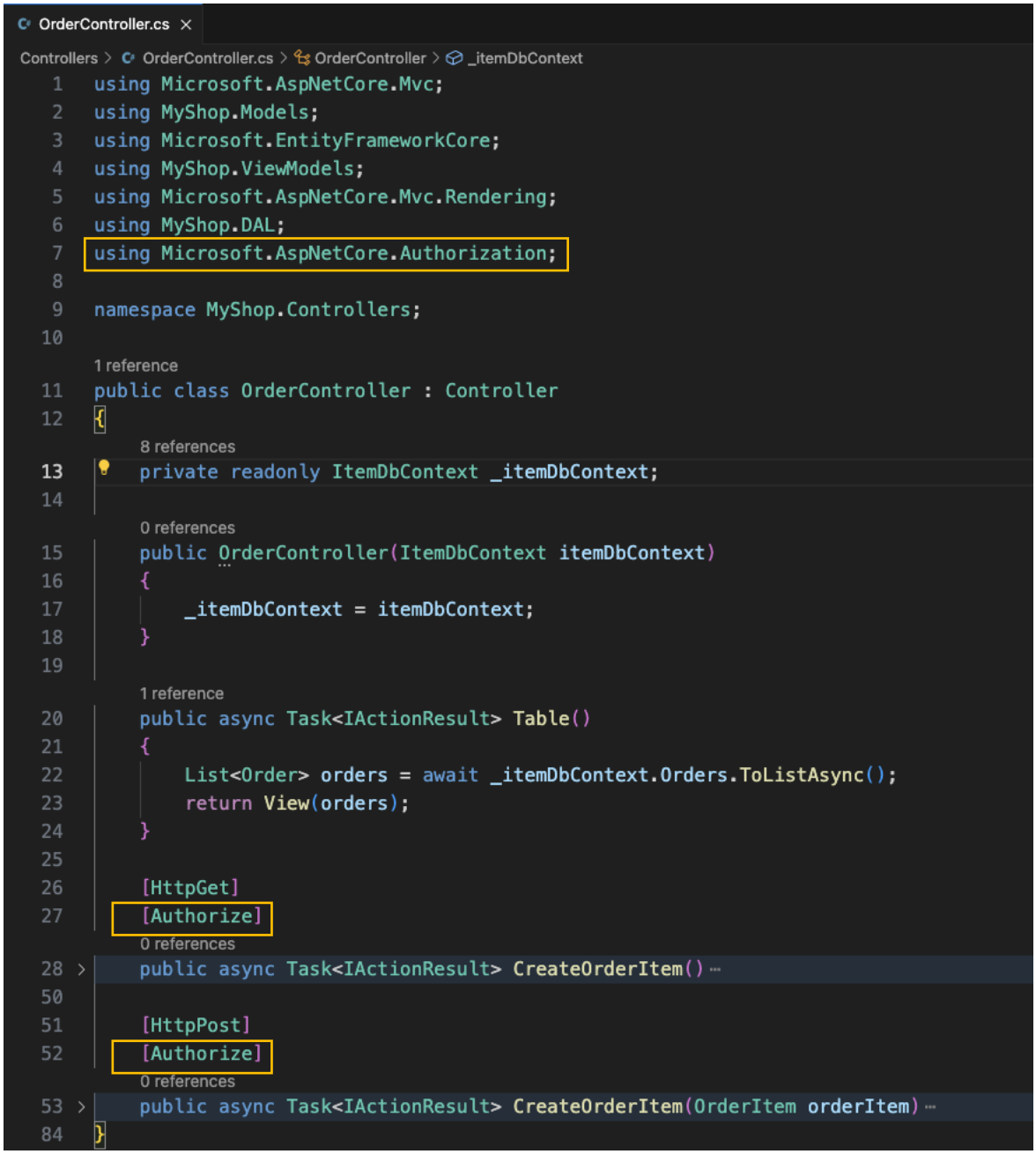
Add Authorisation

You can notice that the users are still able to access all pages without being logged. To prevent this, we simply add a decorator to the controller where the pages are called. We add the decorator [Authorize] to any methods that will change the database to the Controllers/ItemController.cs.

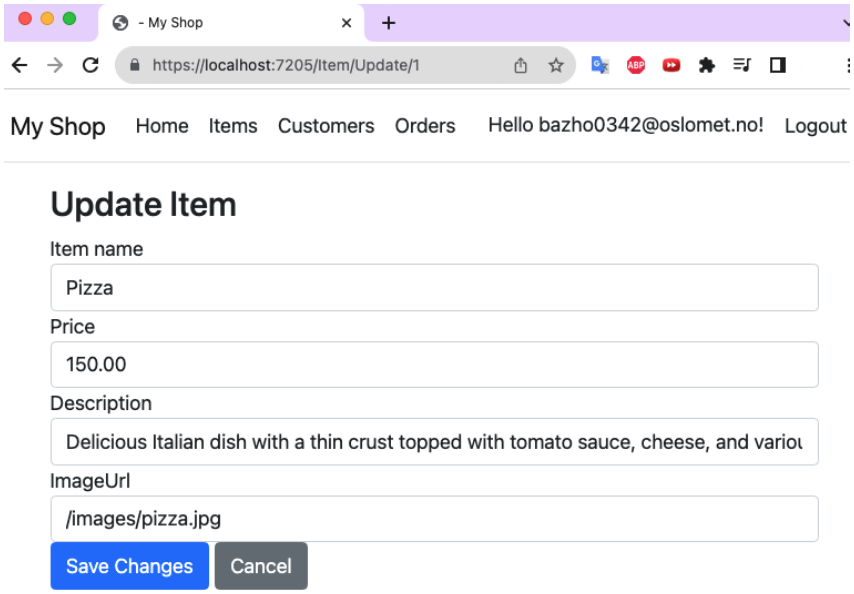
Line1 is required for the decorator.



Similarly, we also add the decorator to the OrderController.cs



Run the project. You can still read all the content without logged in.
But once you try to change the database, the browser will jump to the login page.
Once you are logged in, you will be able to do any change:


- My Shop
https://localhost:7205/Item/Update/1
My Shop
Home
Items
Customers
Orders
Hello bazho0342@oslomet.no!
Logout
Update Item
Item name
Pizza
Price
150.00
Description
Delicious Italian dish with a thin crust topped with tomato sauce, cheese, and variou
ImageUrl
/images/pizza.jpg
Save Changes
Cancel

Password Security & Architecture:

By combining ASP.NET Core's built-in Identity services with a custom MVC controller, our implementation achieves a balance of security and maintainable code:

- **Cryptographic Hashing & Salting:** Passwords are never stored in plain text. When _userManager.CreateAsync(user, model.Password) executes, Identity applies a secure key-derivation function (PBKDF2 with HMAC-SHA512) alongside a unique, randomly generated **salt** for every account. This neutralises rainbow table attacks and credential dumping.
- **Timing-Attack & Open-Redirect Protection:**
  - _signInManager.PasswordSignInAsync() uses constant-time string comparisons during login verification to prevent timing side-channel attacks.
  - Validating URLs with Url.IsLocalUrl(model.ReturnUrl) ensures users cannot be tricked into post-login redirects to malicious external domains (Open Redirect vulnerability).
- **Controlled & Transparent Code (No Hidden Scaffolding):** Auto-generated scaffolding hides authentication logic inside compiled Razor Pages. Implementing our own AccountController gives us complete visibility over the code, keeping the project lightweight, fully debuggable, and easier to maintain.
- **Separation of Concerns:** Delegating complex security tasks (token generation, password hashing, session cookies) to UserManager and SignInManager frees us to focus on application business logic while strictly following industry-standard security practices.

Read more if you are interested:
https://thecyberserver.com/cryptographic-hashing-and-salting-explained/
https://codewithmukesh.com/blog/custom-user-management-in-aspnet-core/

In addition to that, you can extend to more functionalities, such as configure the email sender, forget password, use credential providers such as Google, Facebook, etc. But this is left for yourself to explore!

Bravo! ☺ You now have a web application with good authentication and authorisation!